Phase 3 | Project Design Phase

Solution Architecture

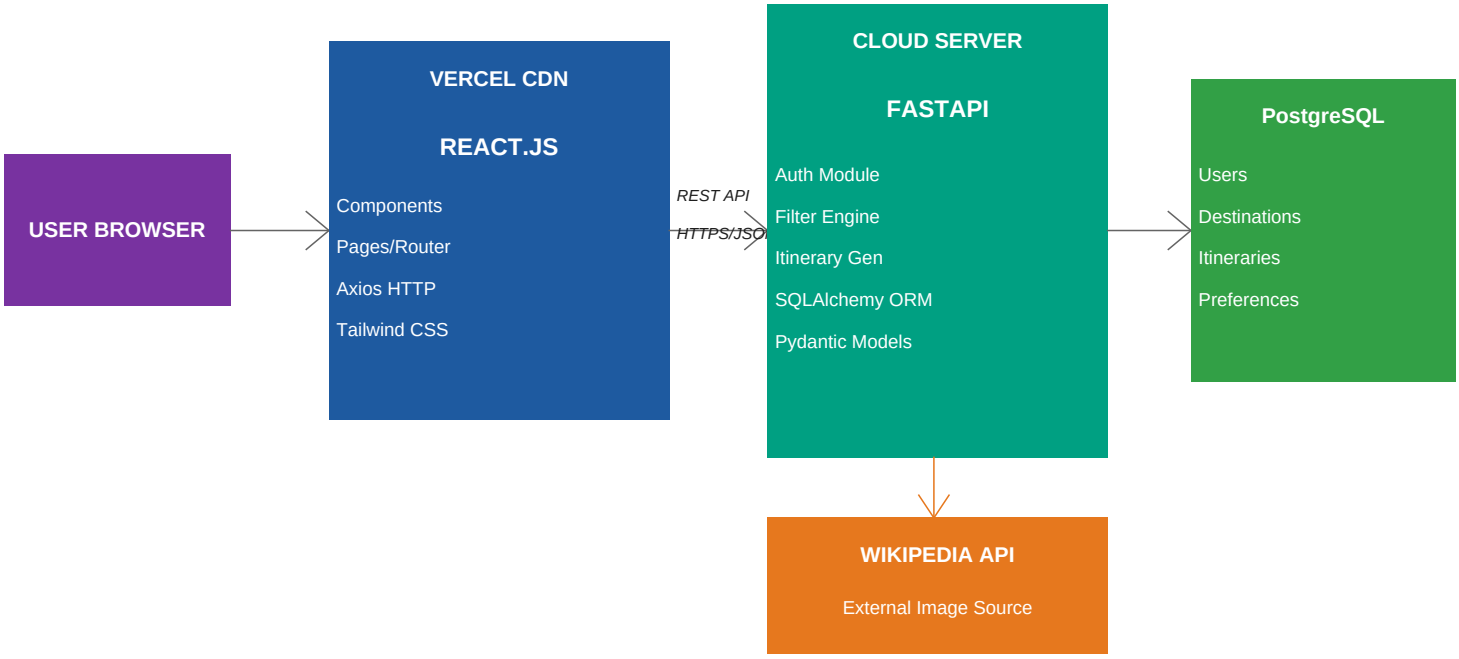
AI-Powered Travel Planner | Smart Situational Itinerary Engine

Live URL:	https://travel-planner-iota-plum.vercel.
GitHub:	github.com/itskian2007/travel-planner
Tech:	React + FastAPI + PostgreSQL + Vercel
Team:	itskian2007

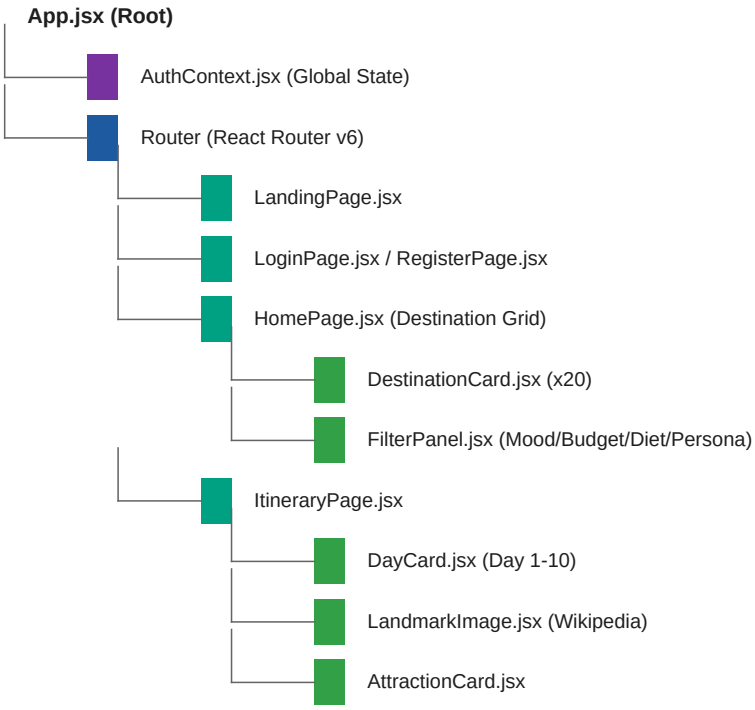
1. Architecture Overview

The Travel Planner employs a 3-tier client-server architecture with a React SPA frontend, FastAPI REST backend, and PostgreSQL database. An external Wikipedia API provides image data. Vercel serves as the deployment platform with global CDN distribution.

2. Full System Architecture Diagram



3. Frontend Component Tree


App.jsx (Root)
AuthContext.jsx (Global State)
Router (React Router v6)
LandingPage.jsx
LoginPage.jsx / RegisterPage.jsx
HomePage.jsx (Destination Grid)
DestinationCard.jsx (x20)
FilterPanel.jsx (Mood/Budget/Diet/Persona)
ItineraryPage.jsx
DayCard.jsx (Day 1-10)
LandmarkImage.jsx (Wikipedia)
AttractionCard.jsx

4. Backend Module Architecture

Module / File	Responsibility
main.py	FastAPI app entry point, CORS config, router registration
routers/auth.py	POST /register, POST /login, POST /forgot-password endpoints
routers/destinations.py	GET /destinations, GET /destinations/{id} endpoints
routers/itinerary.py	POST /itinerary/generate - procedural itinerary engine
routers/images.py	GET /images/{landmark} - Wikipedia API image fetcher
models.py	SQLAlchemy ORM models: User, Destination, Itinerary, Preference
schemas.py	Pydantic request/response validation models
database.py	SQLAlchemy engine, session factory, Base declarative
auth/jwt.py	JWT token creation, decoding, and validation
services/itinerary_engine.py	Core procedural day generation logic
services/image_service.py	Wikipedia API integration, image caching
data/destinations.py	Static data: 20 destinations + curated Day 1-3 itineraries